

TECHIN

JavaScript

Objektai

JavaScript objektai

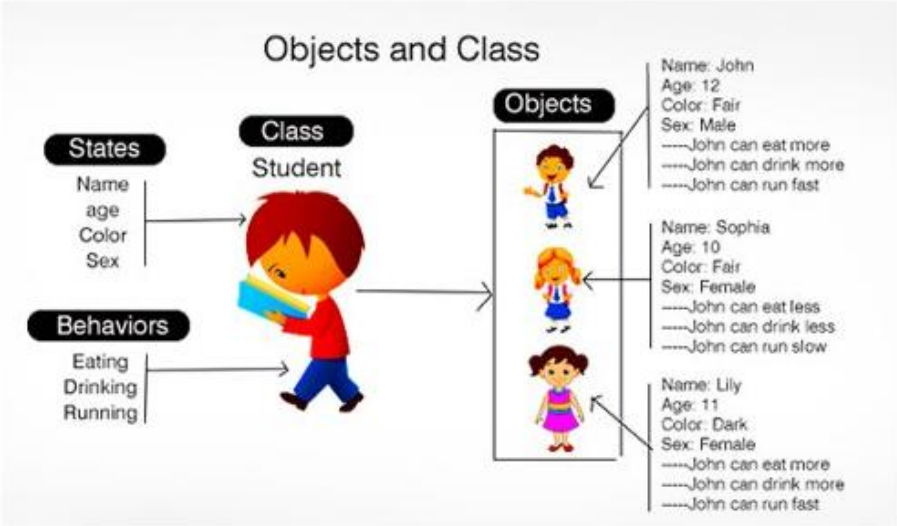
- **Objektas, tai duomenų tipas, skirtas saugoti sudėtingesnės struktūros duomenis.**
- **Objektai programavimo kalboje yra analogai realiame gyvenime esantiems objektams, pvz. automobilis, katė, studentas, segtuvas ir panašiai.**
- **Objektai turi savybes ir metodus.**
- **Savybės apibūdina, koks objektas yra. Pvz. objektas automobilis: koks pavadinimas, kiek metų, kas gamintojas, kokia spalva ir t.t.**
- **Metodai reiškia veiksmą, elgseną ir aprašo, ką objektas moka daryti: judėti, keisti spalvą, kalbėti ir t.t. Metodai aprašomi funkcijomis.**

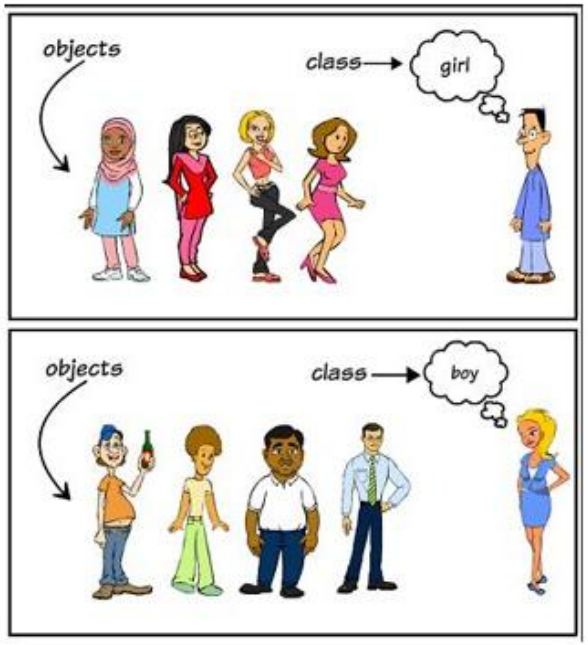
JavaScript objektai

Object	Properties	Methods
	car.name = Fiat car.model = 500 car.weight = 850kg car.color = white	car.start() car.drive() car.brake() car.stop()

JavaScript objektai

Klasė (class) yra ruošinys objektams kurti.





JavaScript objektai

Objektus JavaScript kalboje galime kurti 4 būdais:

1. Object literal
2. Su raktažodžiu new
3. Funkcijų pagalba
4. Klasių pagalba



JavaScript objektai

Object Literal

```
var person = {
  firstName: "John",
  lastName : "Doe",
  id       : 5566,
  fullName : function() {
    return this.firstName + " " + this.lastName;
  }
};
```

Raktinis žodis this rodo į metodo savininką, t.y. objektą.



JavaScript paprasto objekto anatomija





Objektai **savybes ir metodus** **paveldi iš prototipų**, kurie pasiekiami per __proto__.

JavaScript objektai

Object Literal objektą galima sukurti iš kintamųjų:

```
const userName = 'John';
const age = 42;
const logedIn = true;

// sukuriame objektą
const myObj = {userName,  age, logedIn};

console.log(myObj);
```

Rezultatas:

```
▼ (3) {userName: "John", age: 42, logedIn:...}
    userName: "John"
    age: 42
    logedIn: true
```



JavaScript objektai

Kūrimas su raktažodžiu new

Sukuriamas tuščias objektas, o paskui jam priskiriamos savybės.

```
<!DOCTYPE html>
<html>

<body>

  <p id="demo"></p>

  <script>
    var person = new Object();
    person.firstName = "John";
    person.lastName = "Doe";
    person.age = 50;
    person.eyeColor = "blue";
    person.speak = function() {
      let sent = `Mano vardas  ${person.firstName}. Taip.`;
      console.log(sent); //arba veiksmas
      return sent; //arba grąžinama reikšmė
    }

    document.getElementById("demo").innerHTML =
      person.firstName + " is " + person.age + " years old." + person.speak();
  </script>

</body>

</html>
```

JavaScript objektai

Funkcijos pagalba

- Funkcijos vardas rašomas iš didžiosios raidės.
- Funkcijos konstruktoriaus pagalba sukuriami to paties tipo objektai. Jų gali būti keletas. Tai senesnis būdas kurti objektus JS kalboje.
- Taip sukurtam objektui negalime pridėti nei metodo nei savybės tiesiogiai: Person.nationality = "English";
- Savybių ir metodų pridėjimui naudojami prototipai.

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Object Constructors</h2>

<p id="demo"></p>

<script>

// Constructor function for Person objects
function Person(first, last, age, eye) {
  this.firstName = first;
  this.lastName = last;
  this.age = age;
  this.eyeColor = eye;
  this.name = function() {
    return this.firstName + " " + this.lastName
  };
}

// Create a Person object
var myFather = new Person("John", "Doe", 50, "blue");

// Display full name
document.getElementById("demo").innerHTML =
"My father is " + myFather.name();

</script>

</body>
</html>
```

JavaScript objektų prototipai

Kai objektai kuriami su funkcija, savybėms ir metodams pridėti konstruktoriams naudojami Prototipai. Visi objektai paveldi savybes iš prototipo. Prototipas yra objekto pirmtakas.

```
<script>
function Person(first, last, age, eye) {
  this.firstName = first;
  this.lastName = last;
  this.age = age;
  this.eyeColor = eye;
}

Person.prototype.nationality = "English";

var myFather = new Person("John", "Doe", 50, "blue");
document.getElementById("demo").innerHTML =
"The nationality of my father is " + myFather.nationality;
</script>
```

```
<script>
function Person(first, last, age, eye) {
  this.firstName = first;
  this.lastName = last;
  this.age = age;
  this.eyeColor = eye;
}

Person.prototype.name = function() {
  return this.firstName + " " + this.lastName
};

var myFather = new Person("John", "Doe", 50, "blue");
document.getElementById("demo").innerHTML =
"My father is " + myFather.name();
</script>
```

```
▼Person {firstName: "John", lastName: "Doe", age: 50, eyeColor: "blue"}
   firstName: "John"
   lastName: "Doe"
   age: 50
   eyeColor: "blue"
  ▼__proto__:
     nationality: "English"
    ▶constructor: ƒ Person(first, last, age, eye)
    ▶__proto__: Object
```

JavaScript objektai

Klasės pagalba

- JavaScript klasės – naujausias būdas objektams kurti.
- Klasė tai šablonas, pagal kurį galime sukurti daug to paties tipo objektų.
- Klasės JavaScript yra nuo ES6 versijos.
- Tai šiuolaikinis ir modernus būdas kurti objektus.
- Klasės vardas rašomas iš didžiosios raidės.

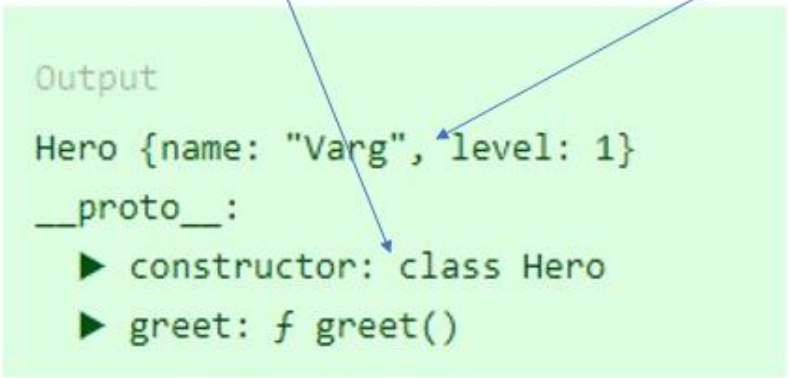
```
1  class User {
2
3    constructor(name) {
4      this.name = name;
5    }
6
7    sayHi() {
8      alert(this.name);
9    }
10
11 }
12
13 // Usage:
14 let user = new User("John");
15 user.sayHi();
```



JavaScript objektai

Kai objektą kuriame iš klasės, jo sudėtį galime pamatyti išvedę į konsolę sukurtą objektą.

console.log(hero1);
Konsolėje, kur matomas išvestas objektas matome: savybes, konstruktorių ir metodus (prototype).



JavaScript darbas su objektai

Objektų savybių ir metodų iškvietimas:

```
const gimli = {
    name: "Gimli",
    race: "dwarf",
    weapon: "axe",
    greet: function() {
        return `Hi, my name is ${this.name}!`;
    },
};
```

Savybių iškvietimas:

```
// Retrieve the value of the weapon property
gimli.weapon;
```

```
Output
"axe"
```

Metodų iškvietimas:

```
gimli.greet();
```

```
Output
"Hi, my name is Gimli!"
```

JavaScript darbas su objektais

Objektų savybių ir metodų pridėjimas ir keitimas ir šalinimas:

Pridėjimas:

```
// Add new age property to gimli
gimli.age = 139;
```

```
// Add new fight method to gimli
gimli.fight = function() {
    return `Gimli attacks with an ${this.weapon}.`;
}
```

Savybių keitimas:

```
// Update weapon from axe to battle axe
gimli.weapon = "battle axe";
```

Šiuo būdu keičiamos tik savybės.

Savybių šalinimas:

```
// Remove weapon from gimli
delete gimli.weapon;
```



JavaScript darbas su objektai

Objektų savybių iškvietimas ir pakeitimas galimas ir per laužtinius skliaustus:

```
const myCar={
    make: "BMW",
    model: "X1",
    year: 2000
}
```

```
myCar['make'] = 'Ford';
myCar['model'] = 'Mustang';
myCar['year'] = 1969;
```

JavaScript darbas su objektai

Objekto savybių pakeitimas, nurodant savybes per kintamuosius.

```
const myCar = {
    make: "Audi",
    model: "Mustang",
    year: 2000
}
```

```
// Pakeičiame savybes nurodant jas per kintamuosius
let propertyName = "make";
myCar[propertyName] = "Ford";   // Audi -> Ford

propertyName = "model";
myCar[propertyName] = "Focus";  // Mustang -> Focus

propertyName = "year";
myCar[propertyName] = 2010;      // 2000 -> 2010

console.log(myCar);
// { make: 'Ford', model: 'Focus', year: 2010 }
```



JavaScript objektų metodas Object.keys()

- **Sukuria objektų savybių pavadinimų (keys) masyvą. Gali būti naudojamas išvesti savybes su pavadinimais (keys and values).**

```
// Initialize an object
const employees = {
    boss: 'Michael',
    secretary: 'Pam',
    sales: 'Jim',
    accountant: 'Oscar'
};

// Get the keys of the object
const keys = Object.keys(employees);

console.log(keys);
```

```
Output
["boss", "secretary", "sales", "accountant"]
```

```
// Iterate through the keys
Object.keys(employees).forEach(key => {
    let value = employees[key];

     console.log(`${key}: ${value}`);
});
```

```
Output
boss: Michael
secretary: Pam
sales: Jim
accountant: Oscar
```

- **Pasinaudojant Object.keys() galima nustatyti objekto dydį - kiek jis turi savybių.**

JavaScript objektų metodas Object.create()

- **Sukuria objektą iš turimo objekto ir susieja jį (perduoda savybes ir metodus) su jau esamu objektu - prototipu.**

```
// Initialize an object with properties and methods
const job = {
    position: 'cashier',
    type: 'hourly',
    isAvailable: true,
    showDetails() {
        const accepting = this.isAvailable ? 'is accepting applications' : "is not currently accepting applications";

        console.log(`The ${this.position} position is ${this.type} and ${accepting}.`);
    }
};

// Use Object.create to pass properties
const barista = Object.create(job);

barista.position = "barista";
barista.showDetails();
```

```
▼Object
   position: "barista"
  ▼__proto__:
     position: "cashier"
     type: "hourly"
     isAvailable: true
   ▶ showDetails: ƒ showDetails()
   ▶ __proto__: Object
```

Objektas barista turi vieną savybę: position, bet visas kitas galima pasiekti per prototipą.



JavaScript objektų metodas Object.values()

- **Sukuria objekto savybių (values) masyvą.**

```
// Initialize an object
const session = {
    id: 1,
    time: `26-July-2018`,
    device: 'mobile',
    browser: 'Chrome'
};

// Get all values of the object
const values = Object.values(session);

console.log(values);
```

```
Output
[1, "26-July-2018", "mobile", "Chrome"]
```

Object.keys() ir Objects.values() yra skirti objekto duomenims paimti.



JavaScript objektų metodas Object.assign()

- **Skirtas objekto duomenims (savybėms ir reikšmėms) nukopijuoti iš vieno objekto į kitą.**

```
// Initialize an object
const name = {
    firstName: 'Philip',
    lastName: 'Fry'
};

// Initialize another object
const details = {
    job: 'Delivery Boy',
    employer: 'Planet Express'
};

// Merge the objects
const character = Object.assign(name, details);

console.log(character);
```

```
Output
{firstName: "Philip", lastName: "Fry", job: "Delivery Boy", employer: "Planet Express"}
```

```
1  // Initialize an object
2  const name = {
3      firstName: 'Philip',
4      lastName: 'Fry'
5  };
6
7  //Kopijuojamos savybės iš name į naują tuščią objektą character
8  const character = Object.assign({}, name);
9  character.nationality = "LT";
10 console.log(character);
```

Objekto kopijavimas į tuščią

JavaScript objektų spread operatorius (...)

```
const circle = {
    radius: 10
};
const coloredCircle = {
    ...circle,
    color: 'black'
};

console.log(coloredCircle);
```

Rezultatas:

```
{
    radius: 10,
    color: 'black'
}
```

JavaScript objektų spread operatorius (...)

Gali būti naudojamas objektų jungimui.

```
const circle = {
    radius: 10
};

const style = {
    backgroundColor: 'red'
};

const solidCircle = {
    ...circle,
    ...style
};

console.log(solidCircle);
```



JavaScript spread perduodant funkcijoms parametrus

```
const numbers = [1, 3, 5, 7];

function addNumbers(a, b, c, d) {
  return a + b + c + d;
}

console.log(addNumbers(...numbers));
```

Rezultatas: 16

JavaScript rest operatorius (...)

```
// Use rest to enclose the rest of specific user-supplied values into an array:
function myBio(firstName, lastName, ...otherInfo) {
  return otherInfo;
}

// Invoke myBio function while passing five arguments to its parameters:
myBio("Oluwatobi", "Sofela", "CodeSweetly", "Web Developer", "Male");

// The invocation above will return:
["CodeSweetly", "Web Developer", "Male"]
```

Naudojamas paduodant parametrus funkcijai.

Trys taškiukai prieš atskirus parametrus funkcijoje juos sujungia į masyvą.

